Lista de Prioridades



Lista de Prioridades

Uma ***lista de prioridade*** é uma tabela na qual a cada um de seus dados está associada uma prioridade.

Uma estrutura de dados ***heap*** é uma **lista linear** que pode ser visualizada como uma **árvore binária completa**, com o último nível preenchido da esquerda para a direita.

Lista de Prioridades

Operações

→ Selecionar o dado de maior prioridade

→ Inserir um novo elemento

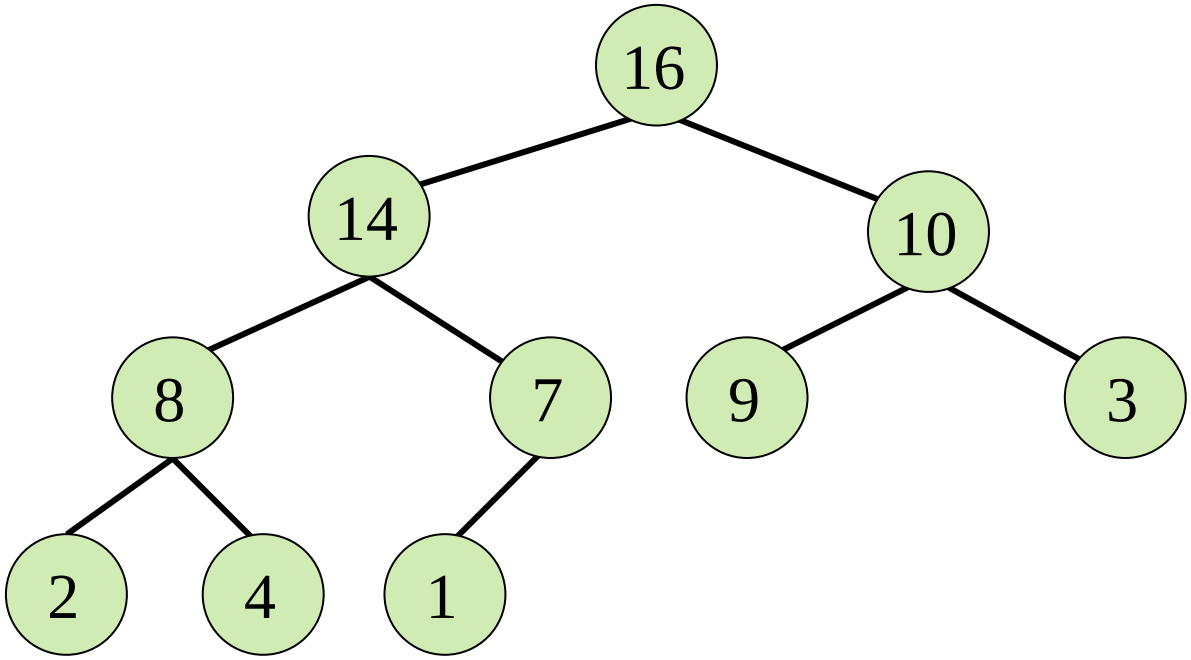
→ Remover o elemento de maior prioridade

Lista de Prioridades

Heap max

16	14	10	8	7	9	3	2	4	1

Armazenamento



Visualização

Lista de Prioridades

Uma *lista de prioridade* é uma lista de elementos com prioridades s_1, ..., s_n, satisfazendo a seguinte propriedade:

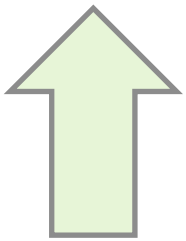
Max

$$s_i \leq s_{\lfloor i/2 \rfloor} \qquad \text{para } 1 \leq i \leq n.$$

Min

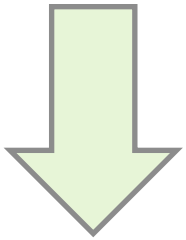
$$s_i \geq s_{\lfloor i/2 \rfloor} \qquad \text{para } 1 \leq i \leq n.$$

Lista de Prioridades



Heap max

16	14	10	8	7	9	3	2	4	1



Heap min

10	14	20	17	27	21	25	19	40	31

Cálculo dos Índices

H é o vetor que armazena a heap.
H tem comprimento *l* e a heap tem comprimento *h_s*,
h_s ≤ *l*.

A **raiz** da árvore é **H[1]**. Dado *i* o índice de um nó, os índices do seu pai e filhos são dados por:

Pai(*i*) = *i* div 2

Filho_esq(*i*) = 2*i*

Filho_dir(*i*) = 2*i* + 1

Exercício

Verifique se as sequências abaixo correspondem a um heap max(min):

a) 20, 25, 32, 29, 27, 35, 40, 45

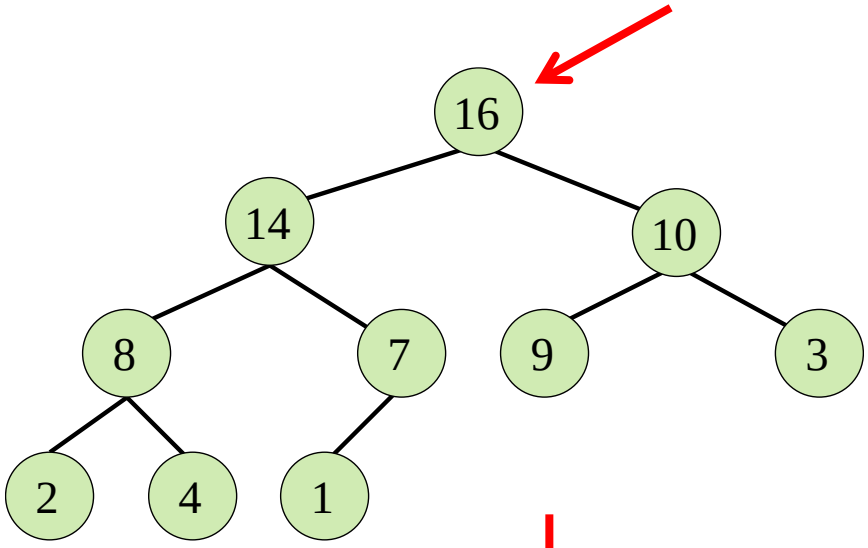
b) 20, 32, 25, 29, 27, 35, 40, 45

c) 50, 45, 48, 29, 15, 35, 40, 27

Remoção

Remove o elemento de maior (menor) prioridade: H[1]

A lista tem que ser ajustada e manter as propriedades



16	14	10	8	7	9	3	2	4	1

Remoção

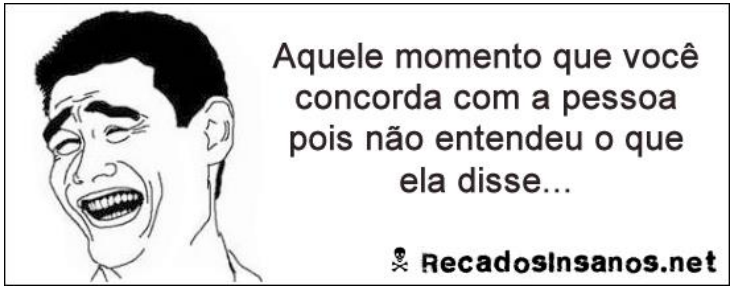
Como fazer isto?



Remoção

Remove o elemento de maior (menor) prioridade: H[1]

O último elemento substitui o primeiro e será usado o procedimento ***descer*** para ajustar este elemento na heap novamente.

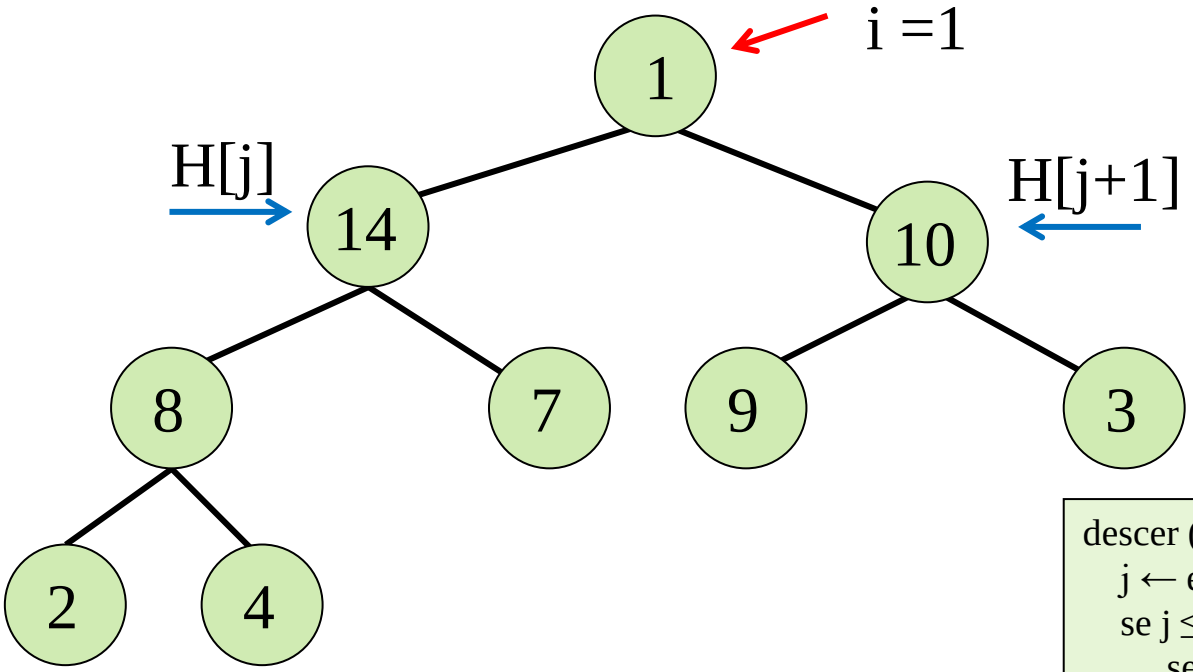


Descer

```
descer (i,n)
    j ← esq(i)
    se j ≤ n então
        se j+1 ≤ n então
            se H[j+1] > H[j] então j ← j + 1
        se H[i] < H[j] então
            troca(i,j)
            descer(j,n)
```

Verifica filho de maior prioridade

Descer



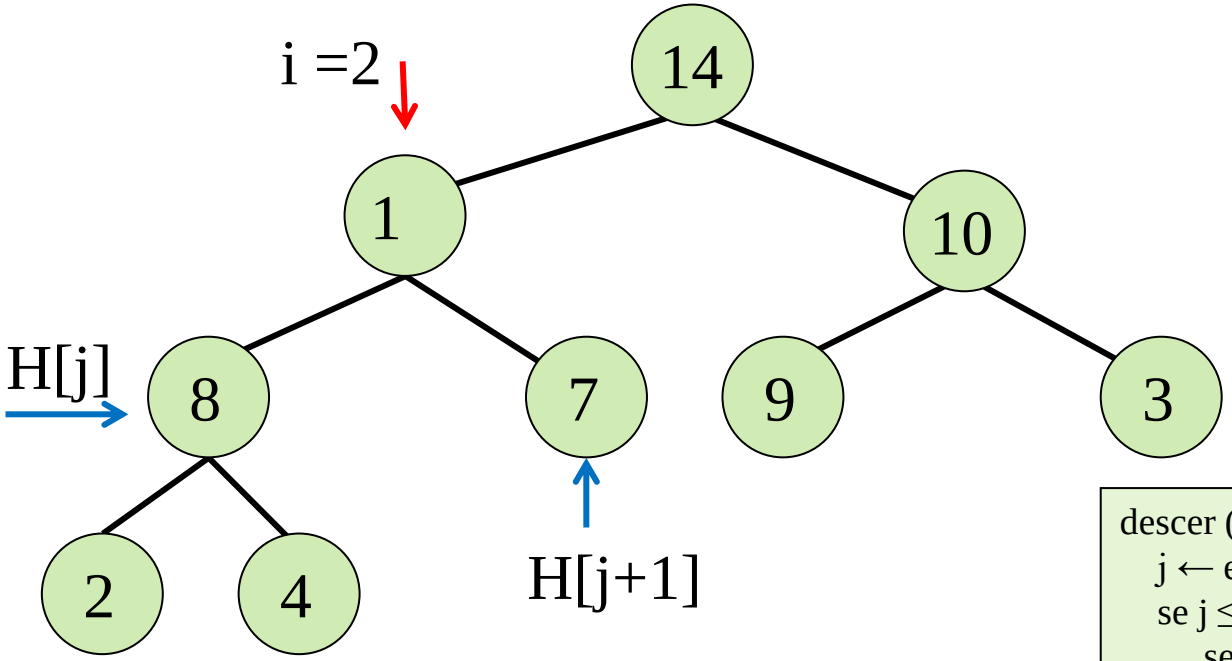

```
descer (i,n)
    j ← esq(i)
    se j ≤ n então
        se j+1 ≤ n então
            se H[j+1] > H[j] então j ← j + 1
        se H[i] < H[j] então
            troca(i,j)
            descer(j,n)
```

j = 2

1	14	10	8	7	9	3	2	4	

n =9

Descer



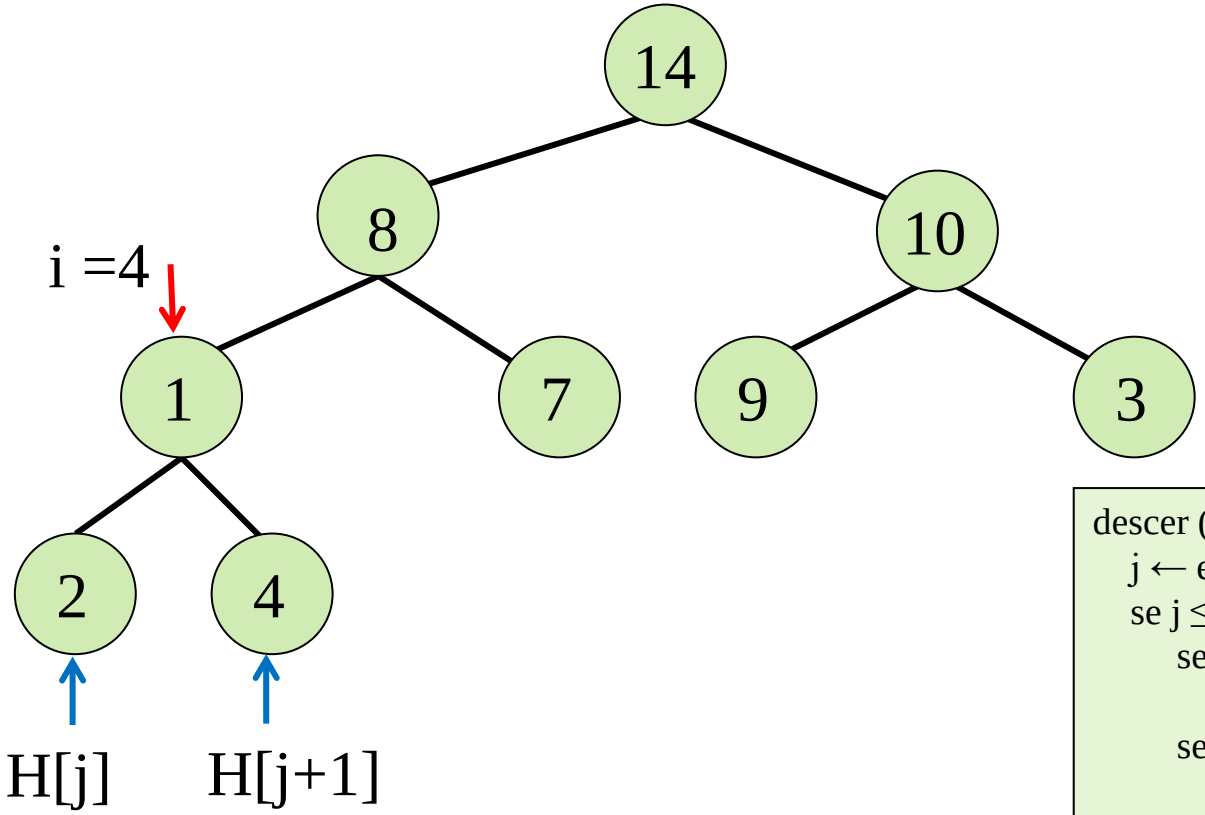

```
descer (i,n)
    j ← esq(i)
    se j ≤ n então
        se j+1 ≤ n então
            se H[j+1] > H[j] então j ← j + 1
        se H[i] < H[j] então
            troca(i,j)
            descer(j,n)
```

j = 4

14	1	10	8	7	9	3	2	4	

n =9

Descer



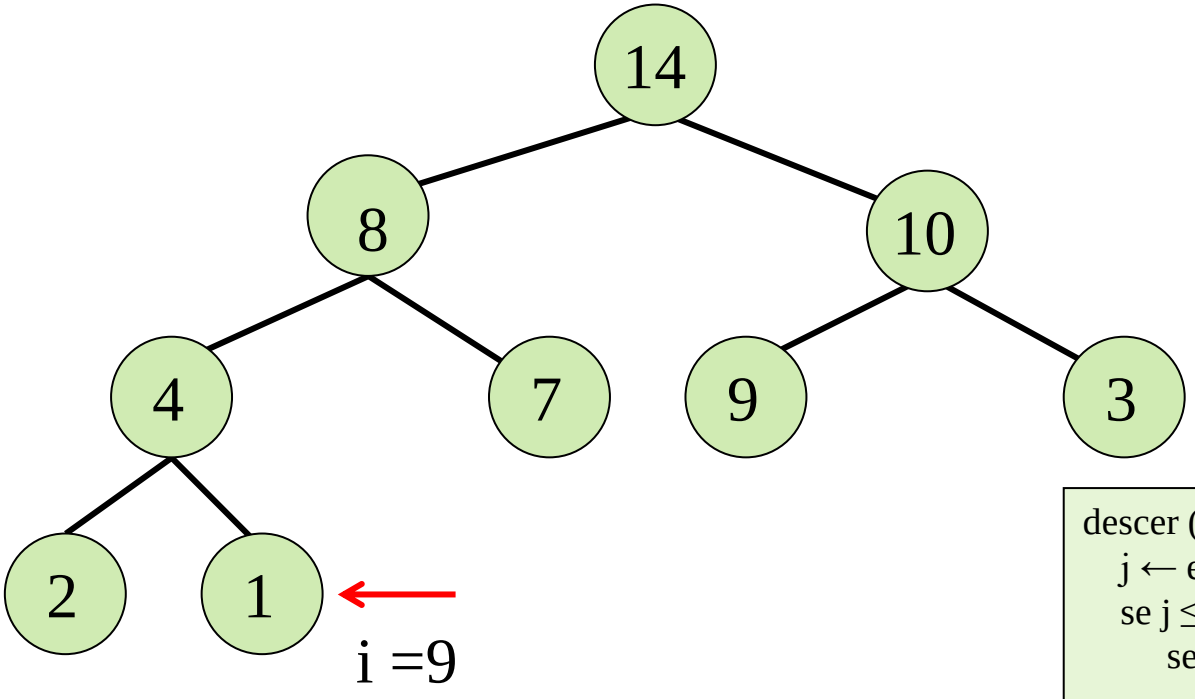

```
descer (i,n)
    j ← esq(i)
    se j ≤ n então
        se j+1 ≤ n então
            se H[j+1] > H[j] então j ← j + 1
        se H[i] < H[j] então
            troca(i,j)
            descer(j,n)
```

14	8	10	1	7	9	3	2	4	

j = 9

n =9

Descer




```
descer (i,n)
    j ← esq(i)
    se j ≤ n então
        se j+1 ≤ n então
            se H[j+1] > H[j] então j ← j + 1
        se H[i] < H[j] então
            troca(i,j)
            descer(j,n)
```

j = 18 FIM

14	8	10	4	7	9	3	2	1	

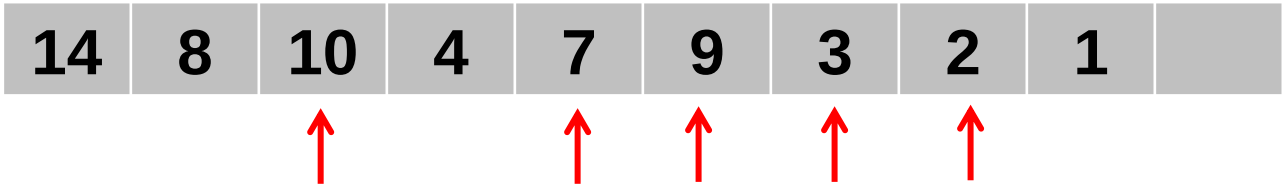
n =9

Descer

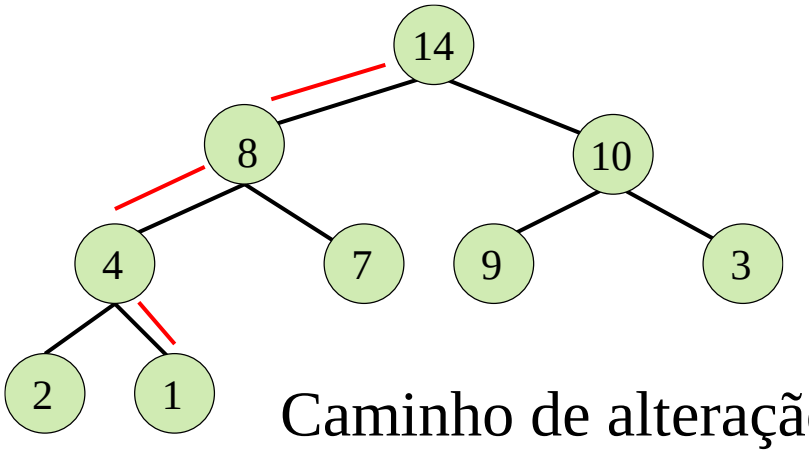
Qual a complexidade de *descer*?



Descer



Posições inalteradas



Caminho de alteração

Altura de uma árvore binária completa

Descer

Qual a complexidade de *descer*?

$O(\log n)$

Remoção

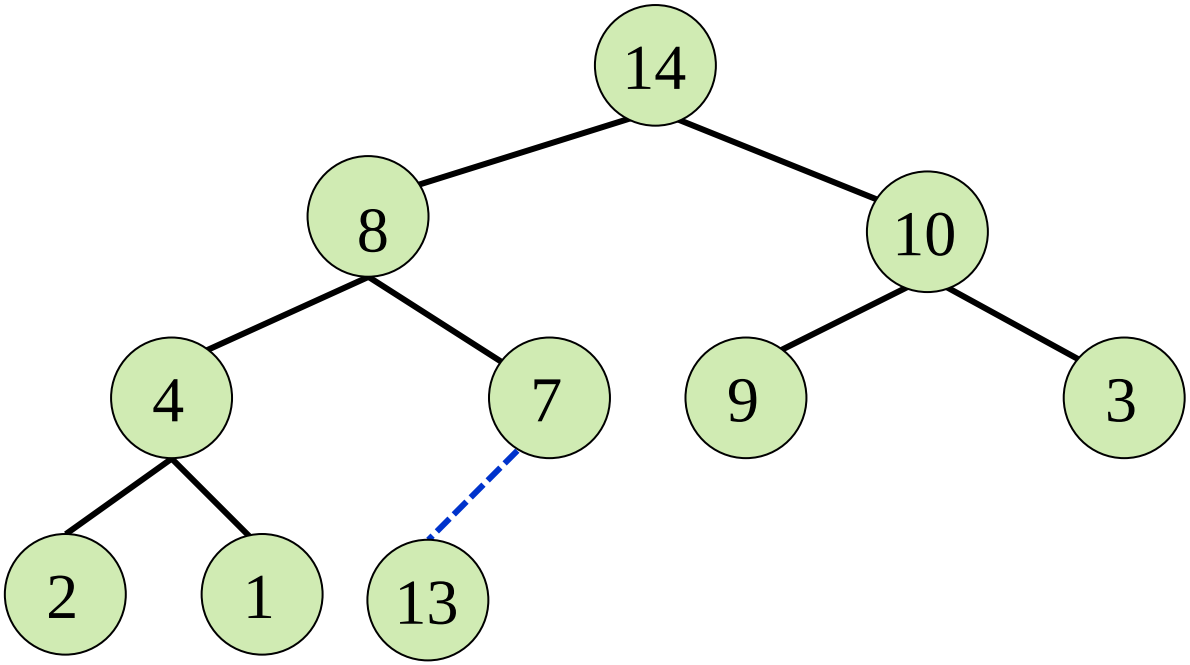
```
Remoção
   se ( n > 0 ) então
      elemento ← H[1]
      H[1] ← H[n]
      n ← n -1
      descer(1,n)
  senão “Heap Vazia”
```

Qual a complexidade da *remoção* do elemento de maior prioridade?

$\Theta(\log n)$

Inserção

O novo dado é inserido na posição ***n*+1** e “subido” se necessário



Posição n+1

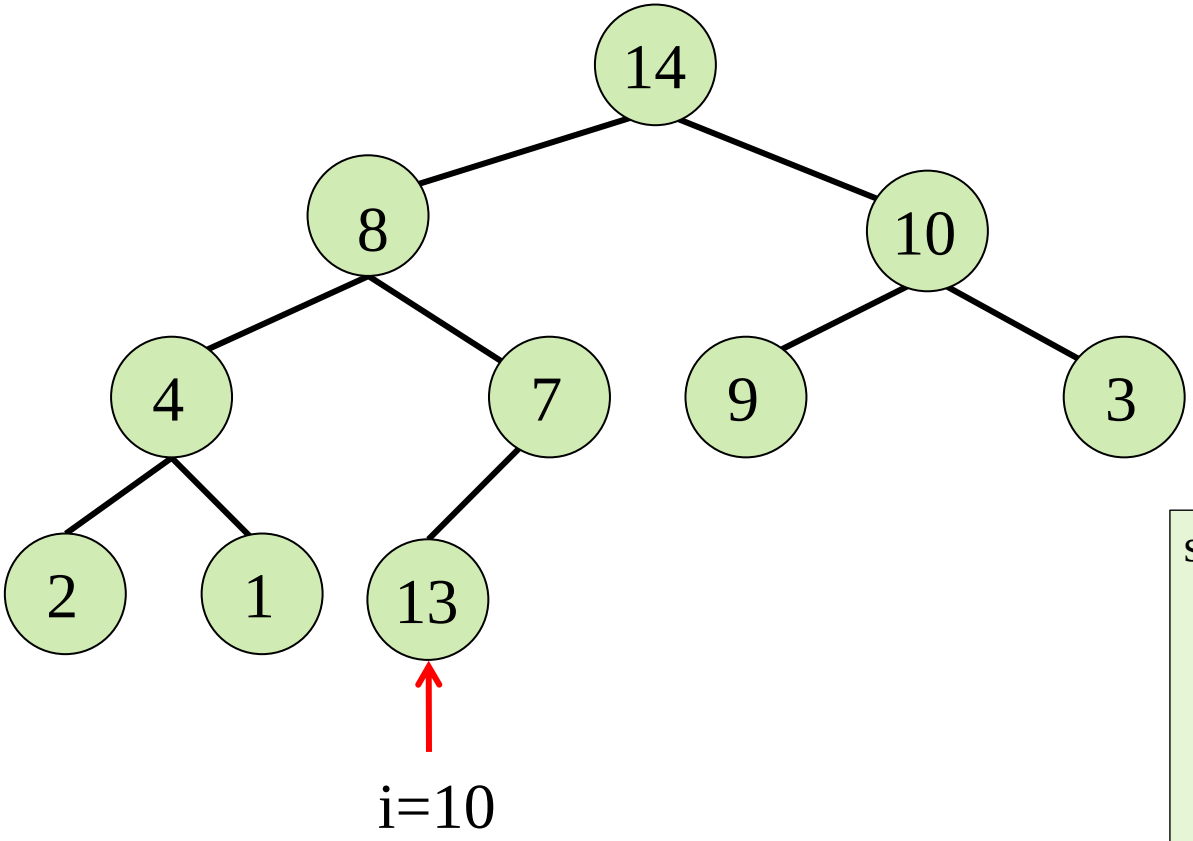


14	8	10	4	7	9	3	2	1	13

Subir

```
subir (i)
    j ← pai(i)
    se j ≥ 1 então
        se H[i] > H[j] então
            troca(i,j)
            subir(j)
```

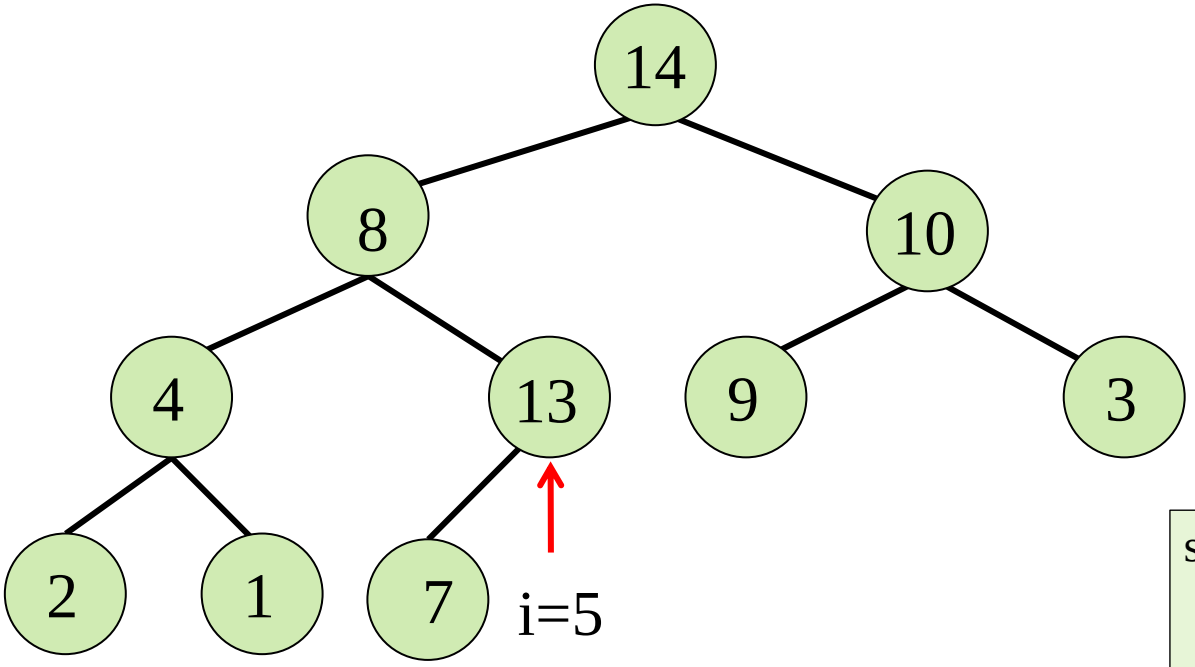
Subir




```
subir (i)
    j ← pai(i)
    se j ≥ 1 então
        se H[i] > H[j] então
            troca(i,j)
            subir(j)
```

14	8	10	4	7	9	3	2	1	13

Subir

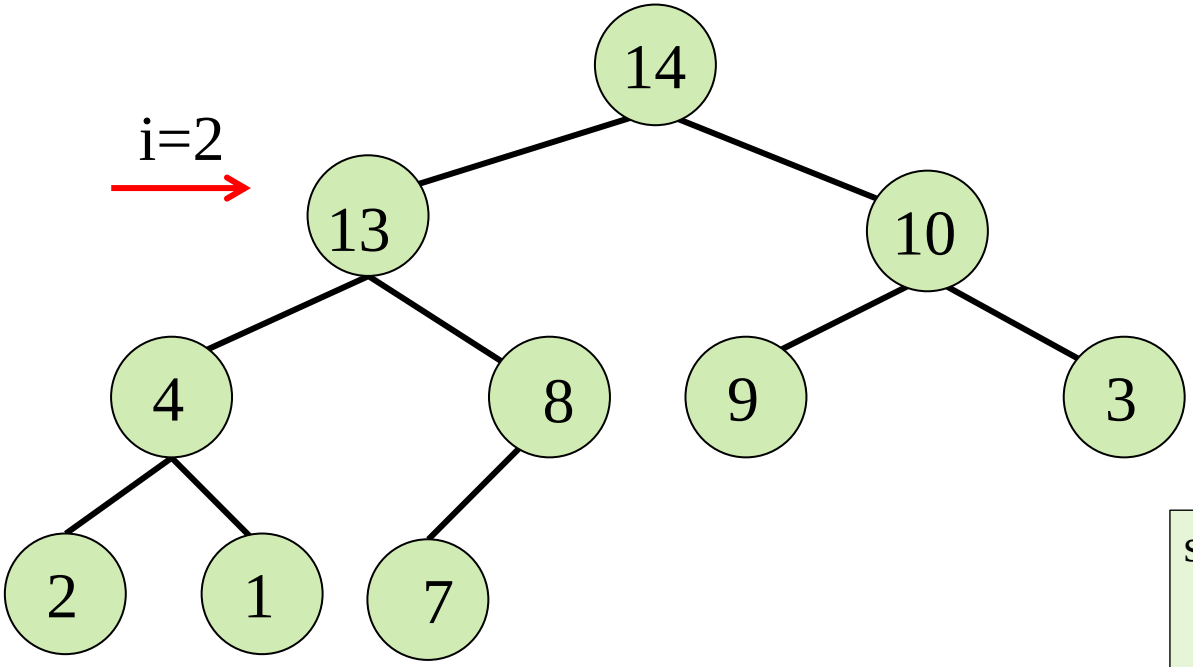



```
subir (i)
    j ← pai(i)
    se j ≥ 1 então
        se H[i] > H[j] então
            troca(i,j)
            subir(j)
```

j=2

14	8	10	4	13	9	3	2	1	7

Subir




```
subir (i)
    j ← pai(i)
    se j ≥ 1 então
        se H[i] > H[j] então
            troca(i,j)
            subir(j)
```

j=1

FIM

14	13	10	4	8	9	3	2	1	7

Inserção

```
Inserção
   se ( n < l ) então
      n ← n + 1
      H[n] ← novo
      subir(n)
   senão "Não há espaço"
```

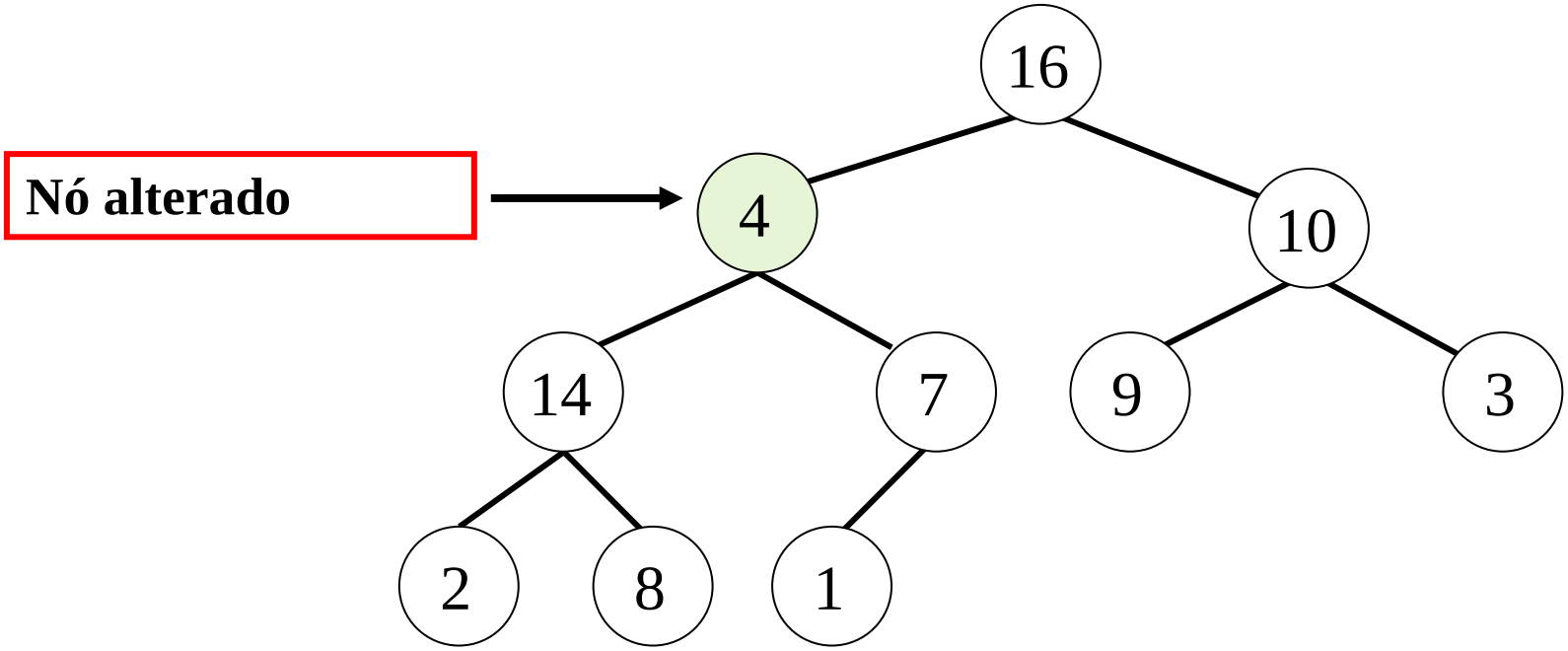
Complexidade da *inserção*: $O(\log n)$

Porque *O*?



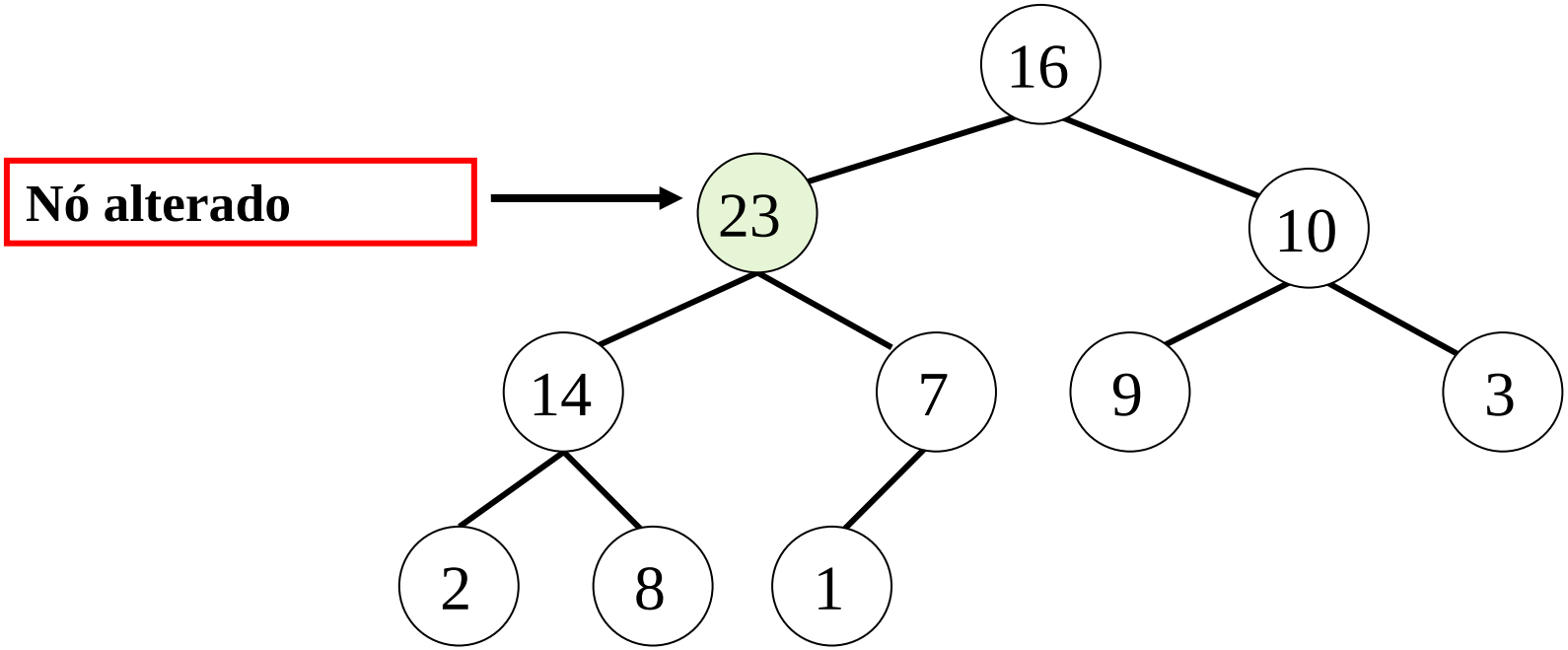
Alteração de Prioridade

Suponha que uma prioridade H[*i*] tenha sido modificada e tenha ficado menor que a prioridade dos seus filhos. Isto contraria a propriedade da heap max. Sendo assim, H[*i*] deve "descer" por um caminho na árvore até que a propriedade seja satisfeita.



Alteração de Prioridade

Suponha que uma prioridade H[*i*] tenha sido modificada e tenha ficado maior que a prioridade do pai. Isto contraria a propriedade da heap max. Sendo assim, H[*i*] deve "subir" por um caminho na árvore até que a propriedade seja satisfeita.



Alteração de Prioridade

Fazer o algoritmo de alteração de prioridade.

Qual a complexidade do seu algoritmo?

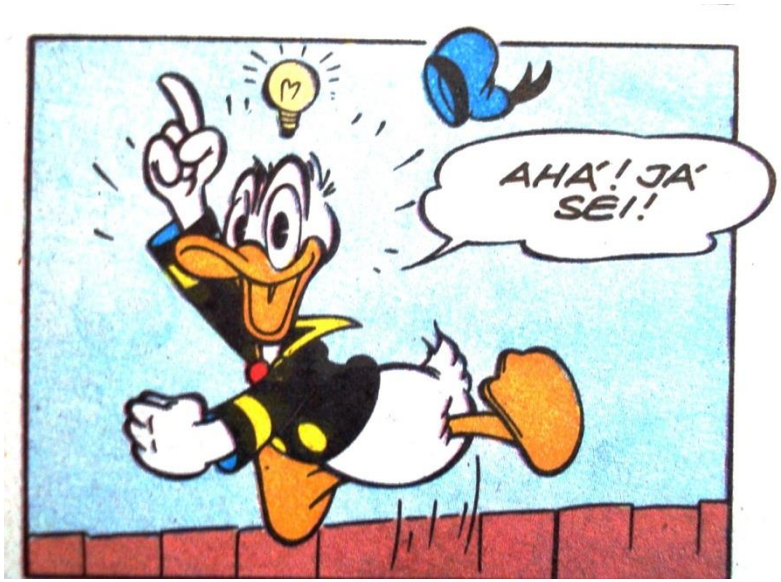


Construção

Dado um vetor com *n* elementos dispostos de modo qualquer, como construir uma *heap*?



Construção



Ideia 1: Olhar os elementos um a um e considerar uma nova inserção.

Algoritmo
    para i ← 2 até *n* faça
        subir(i)

→ **Complexidade: $O(n \log n)$**

Construção



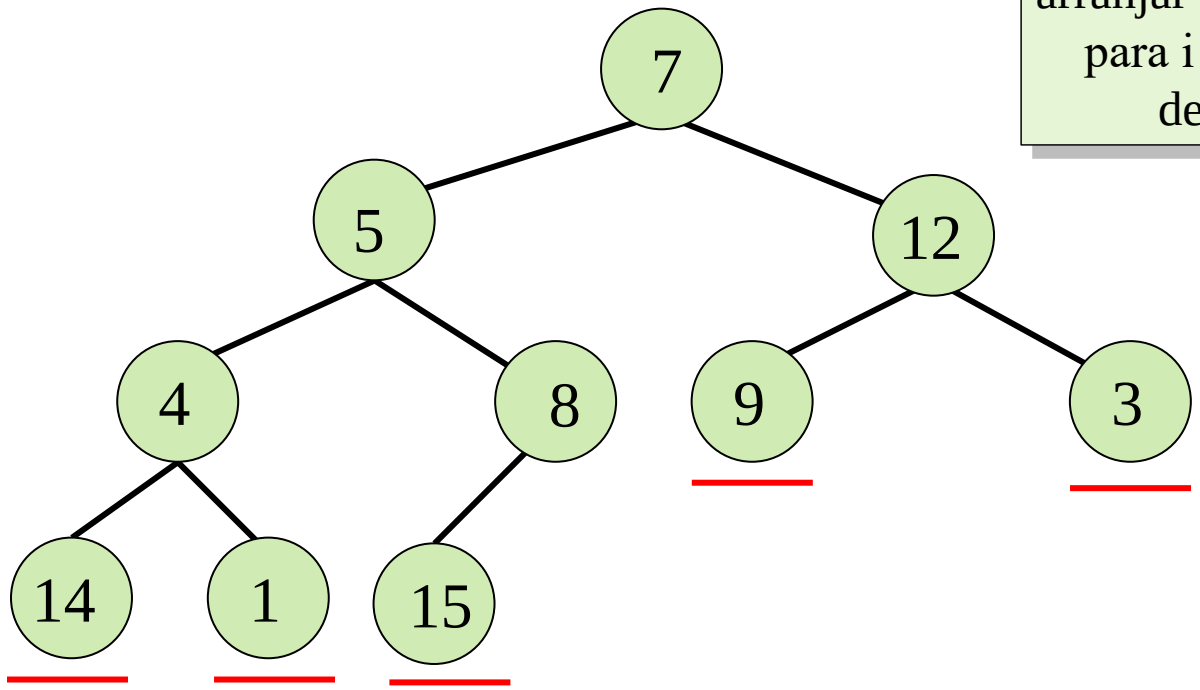
Ideia 2: Uma vez que $\lfloor n/2 \rfloor + 1$ elementos são folhas, não é necessário preocupar-se com tais elementos, dado que a propriedade da heap estará satisfeita para eles. Assim, deve-se, arranjar os $\lfloor n/2 \rfloor$ nós não folha

```
arranjar (n)
    para i ← ⌊n/2⌋ até 1 faça
        descer(i,n)
```

→ **Complexidade: ?**

Construção

```
arranjar (n)
    para i ← ⌊n/2⌋ até 1 faça
        descer(i,n)
```

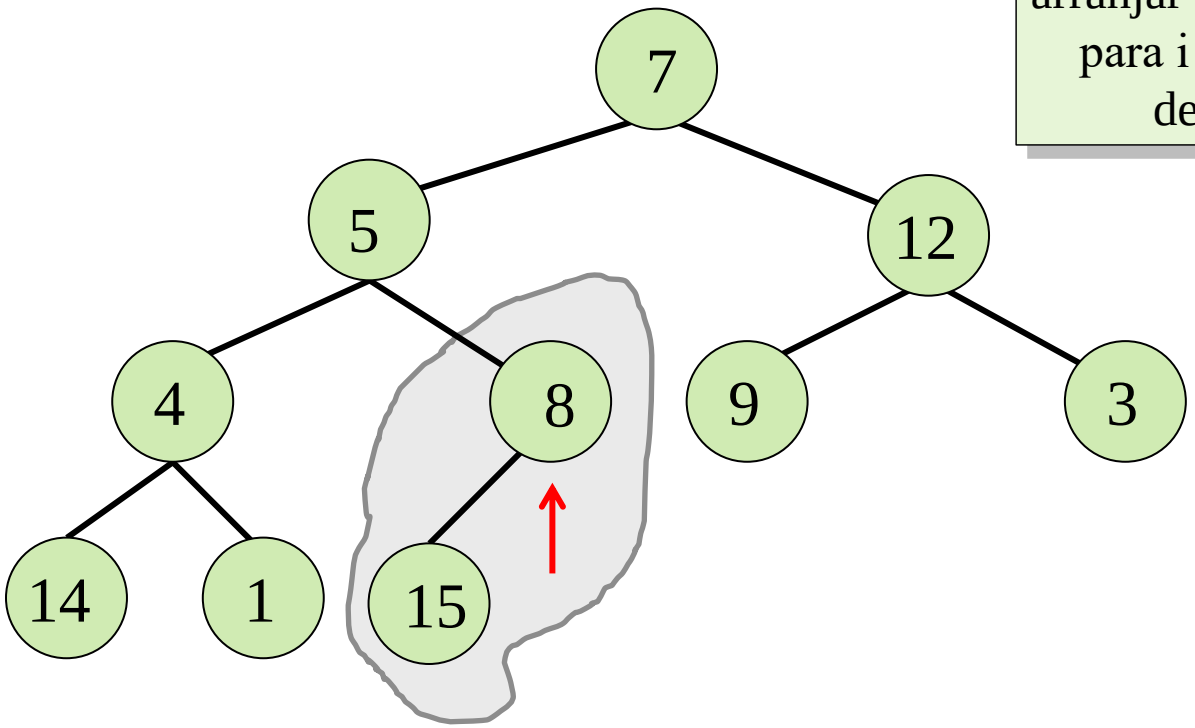




7	5	12	4	8	9	3	14	1	15

Construção

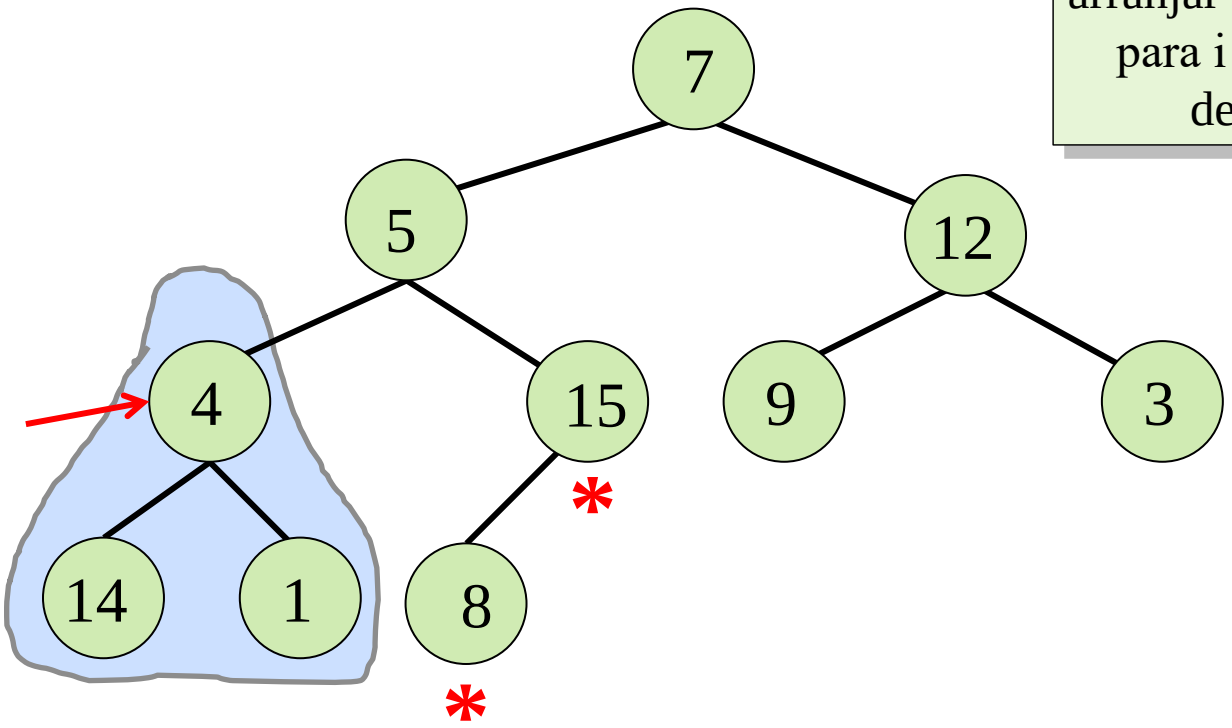
```
arranjar (n)
    para i ← ⌊n/2⌋ até 1 faça
        descer(i,n)
```



7	5	12	4	8	9	3	14	1	15

Construção

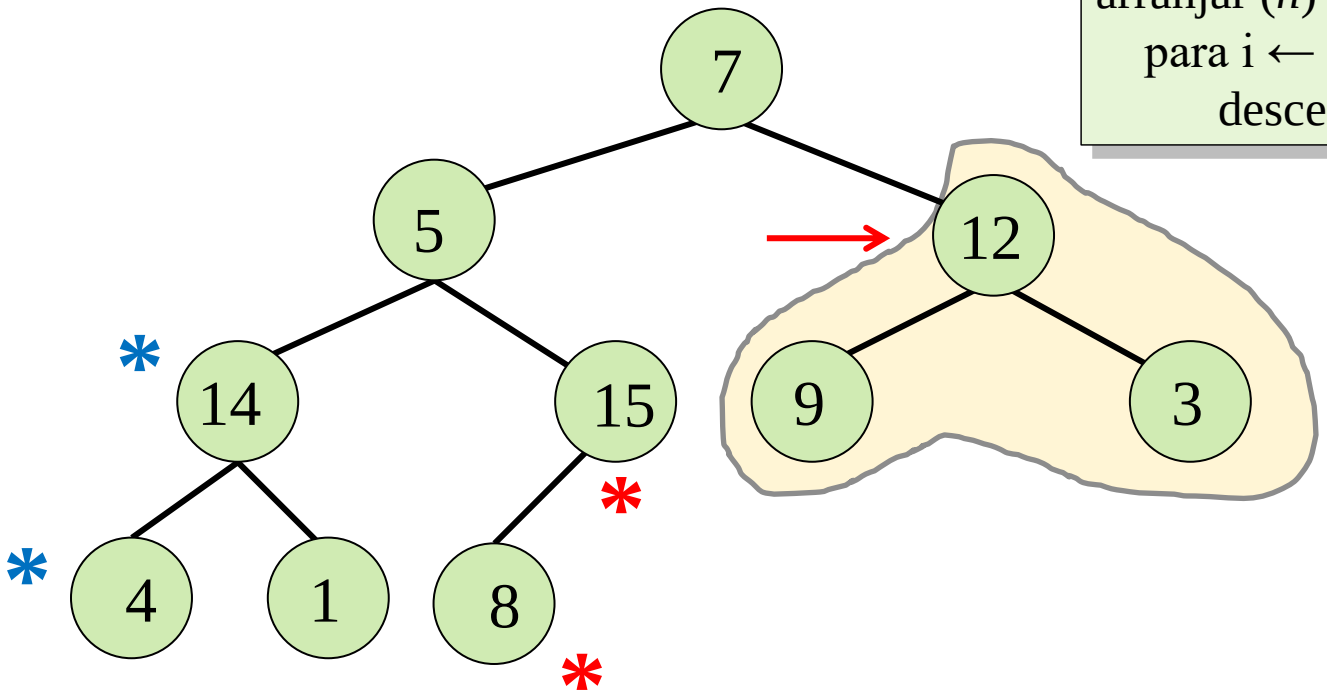
```
arranjar (n)
    para i ← ⌊n/2⌋ até 1 faça
        descer(i,n)
```



7	5	12	4	15	9	3	14	1	8

Construção

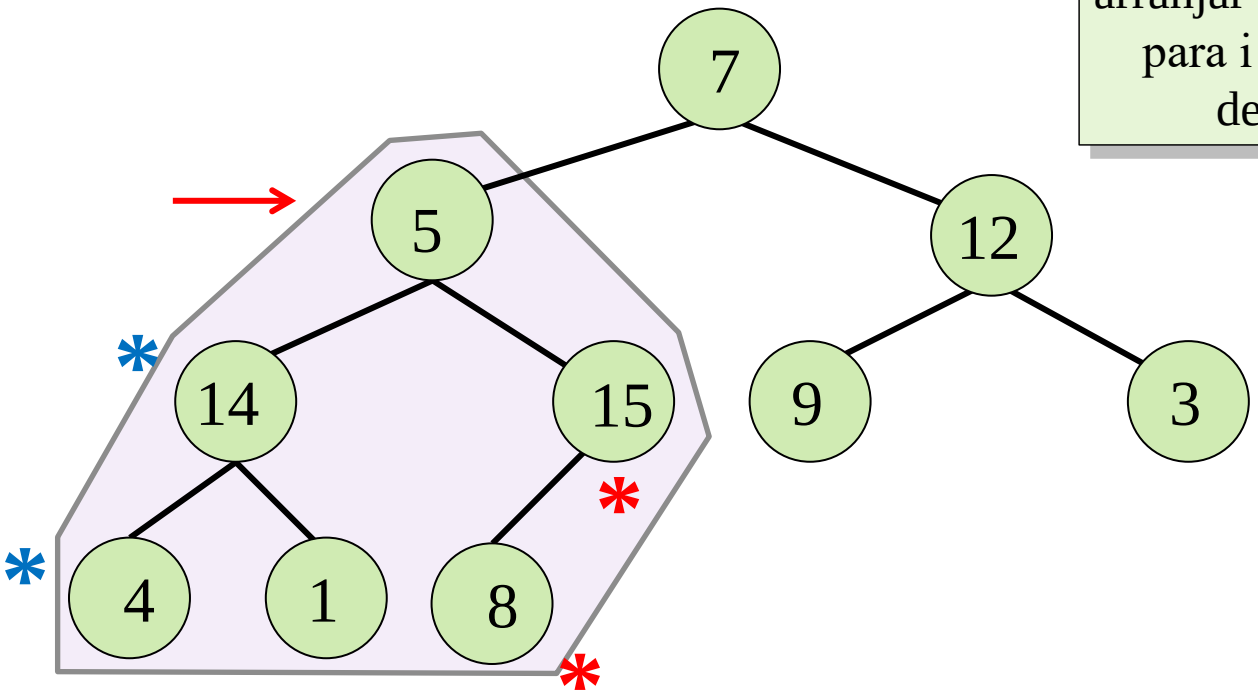
```
arranjar (n)
    para i ← ⌊n/2⌋ até 1 faça
        descer(i,n)
```



7	5	12	14	15	9	3	4	1	8

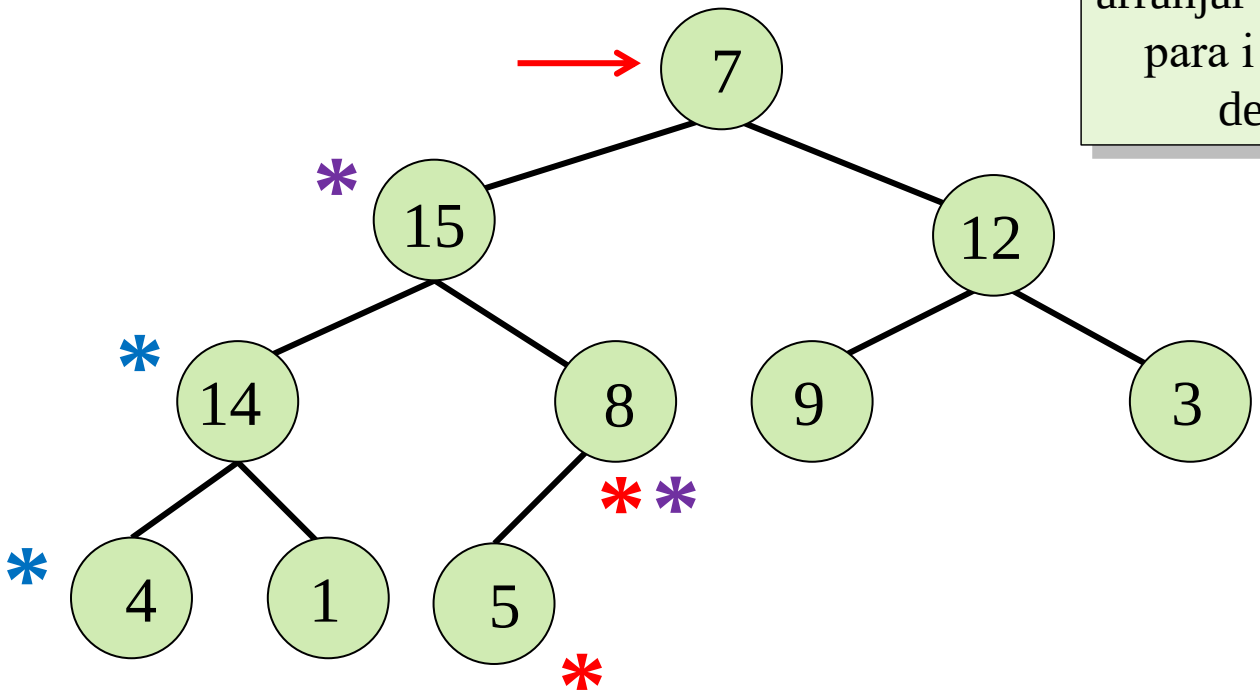
Construção

```
arranjar (n)
    para i ← ⌊n/2⌋ até 1 faça
        descer(i,n)
```



7	5	12	14	15	9	3	4	1	8

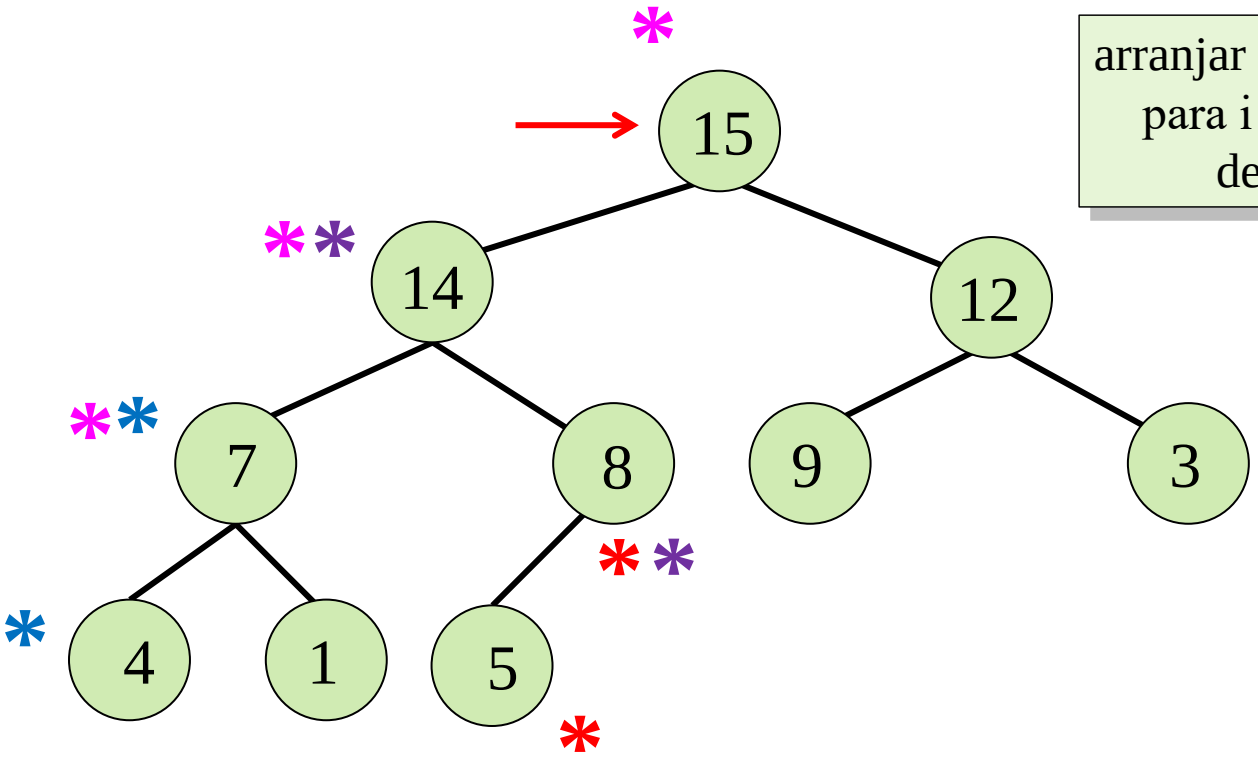
Construção




```
arranjar (n)
    para i ← ⌊n/2⌋ até 1 faça
        descer(i,n)
```

7	15	12	14	8	9	3	4	1	5

Construção




```
arranjar (n)
    para i ← ⌊n/2⌋ até 1 faça
        descer(i,n)
```

15	14	12	7	8	9	3	4	1	5

Construção

Lema. O algoritmo apresentado constrói uma lista de prioridades em tempo linear.

Prova. O tempo requerido pelo procedimento descer(i,*n*) é T(i) = h_i – 1, sendo h_i a altura do nó *i*. Tal procedimento é chamado apenas para nós com altura maior que 1 (metade dos nós da lista).
Observa-se que no máximo $\lceil n/2^j \rceil$ possuem altura *j*.

O tempo despendido é

$$\sum_{j=2}^{h} j \frac{n}{2^j}$$

que é $O(n)$.

Construção

Seja a lista abaixo

40, 25, 72, 13, 14, 95, 48, 100, 12, 93.

Determinar o heap max obtido pelo algoritmo de construção.



Heapsort

1. Organizar os elementos como uma heap
(procedimento arranjar).

2. Uma vez que o elemento de maior prioridade
se encontra na primeira posição, H[1], basta remove-lo (colocando na última posição).

3. Repetir o processo para os $n - 1$ elementos do heap.
Isto deve ser feito até o heap ter tamanho 2.

Heapsort

```
procedimento heapsort
    arranjar (n)
    m ← n
    enquanto (m > 1) faça
       troca(i,m)
       m ← m – 1
       descer(1,m)
```

arranjar (*n*) → $O(n)$

enquanto (*m* > 1) faça → $O(n)$

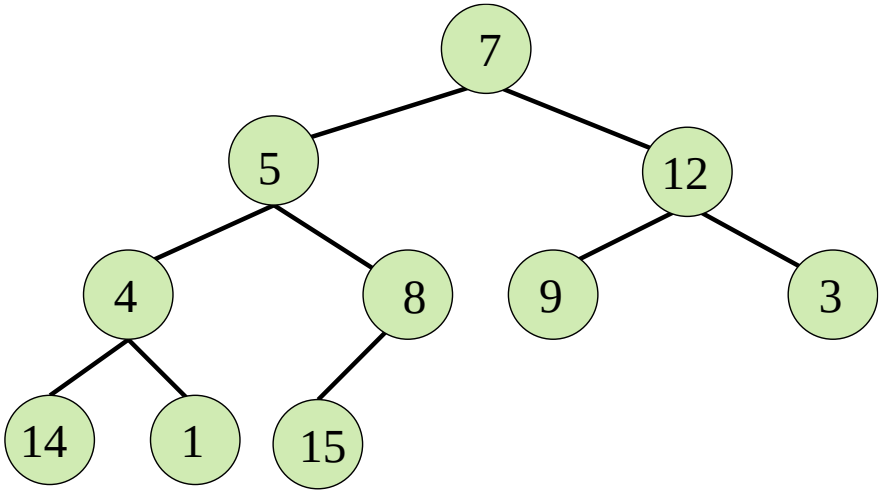
descer(1,*m*) → $O(\log n)$

Complexidade: $O(n \log n)$

Heapsort

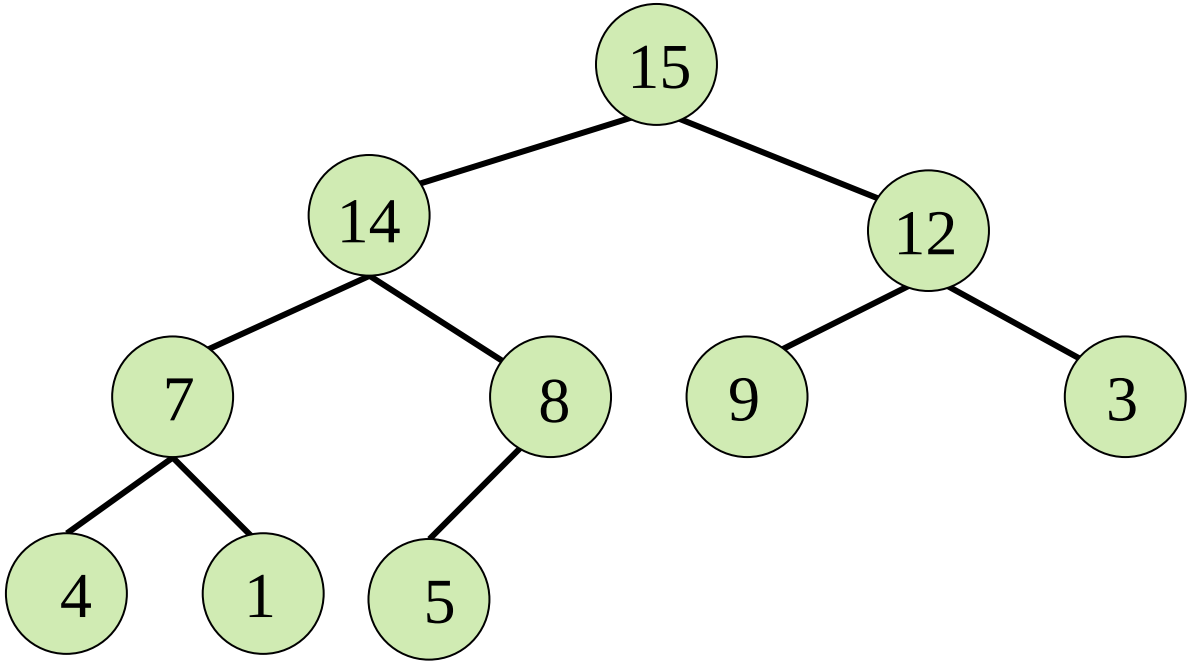
Exemplo: Aplicar Heapsort na seguinte lista.

7	5	12	4	8	9	3	14	1	15



Heapsort

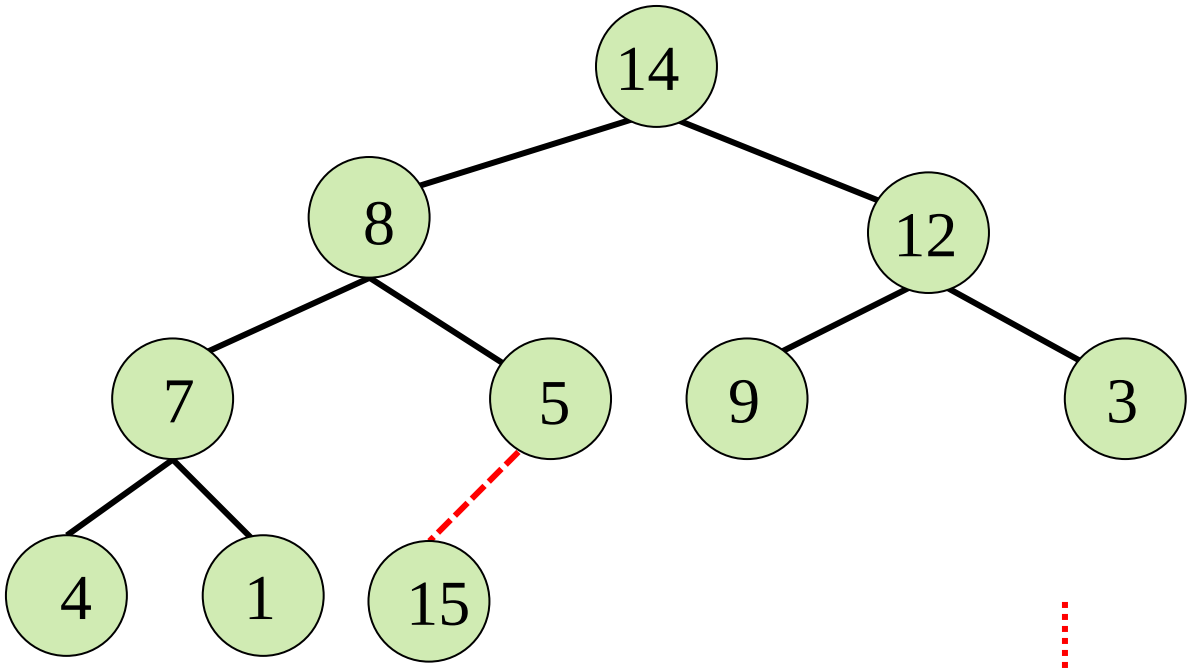
Arranjar(*n*)



15	14	12	7	8	9	3	4	1	5

Heapsort

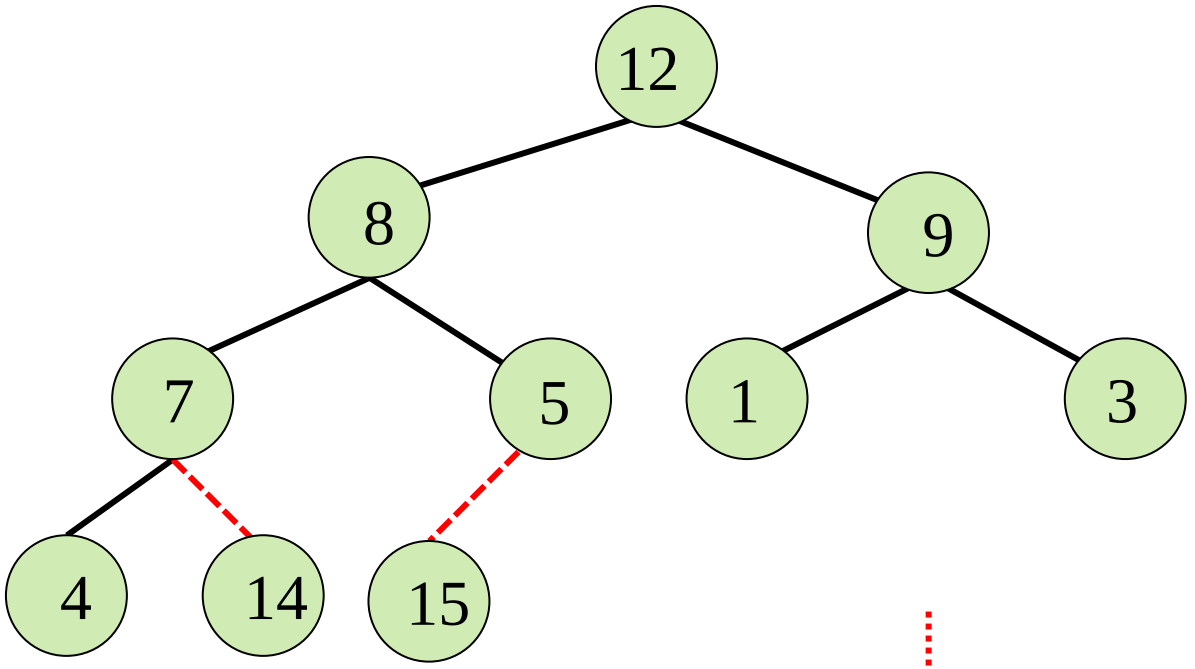
Trocar e descer



14	8	12	7	5	9	3	4	1	15

Heapsort

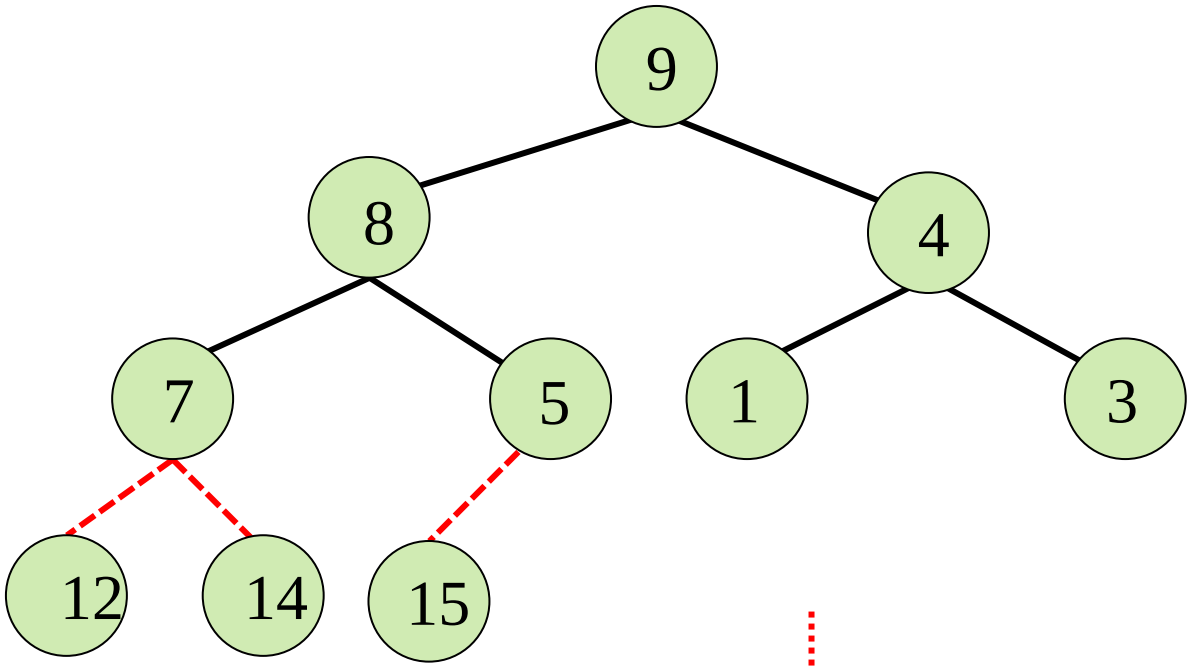
Trocar e descer



12	8	9	7	5	1	3	4	14	15

Heapsort

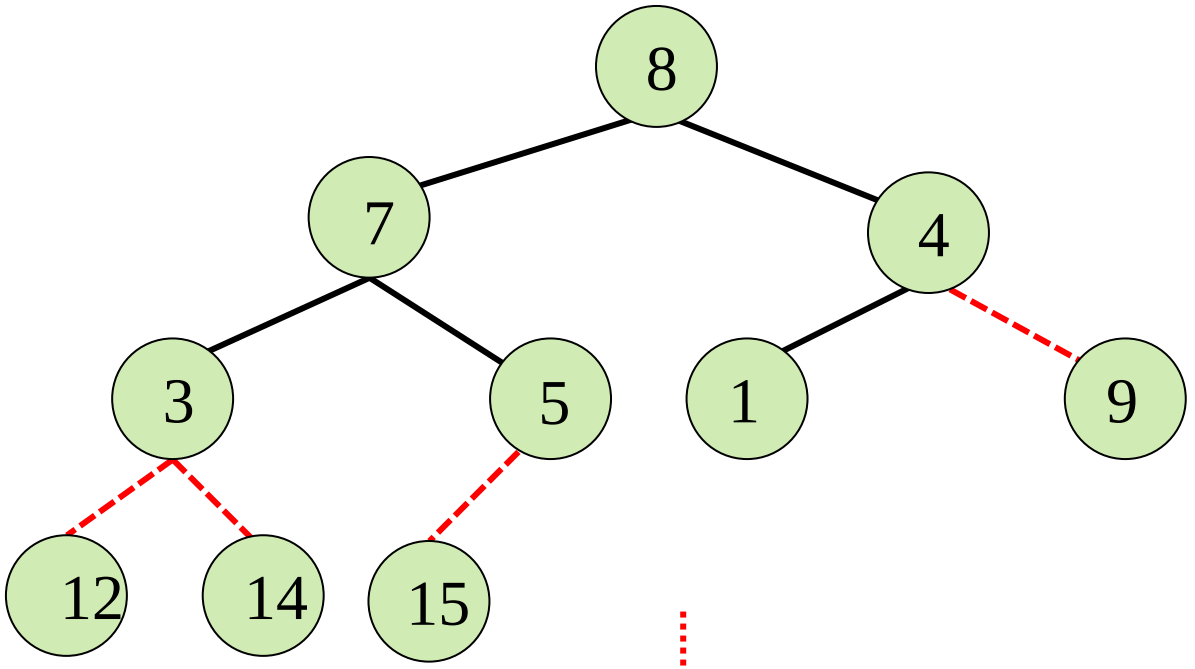
Trocar e descer



9	8	4	7	5	1	3	12	14	15

Heapsort

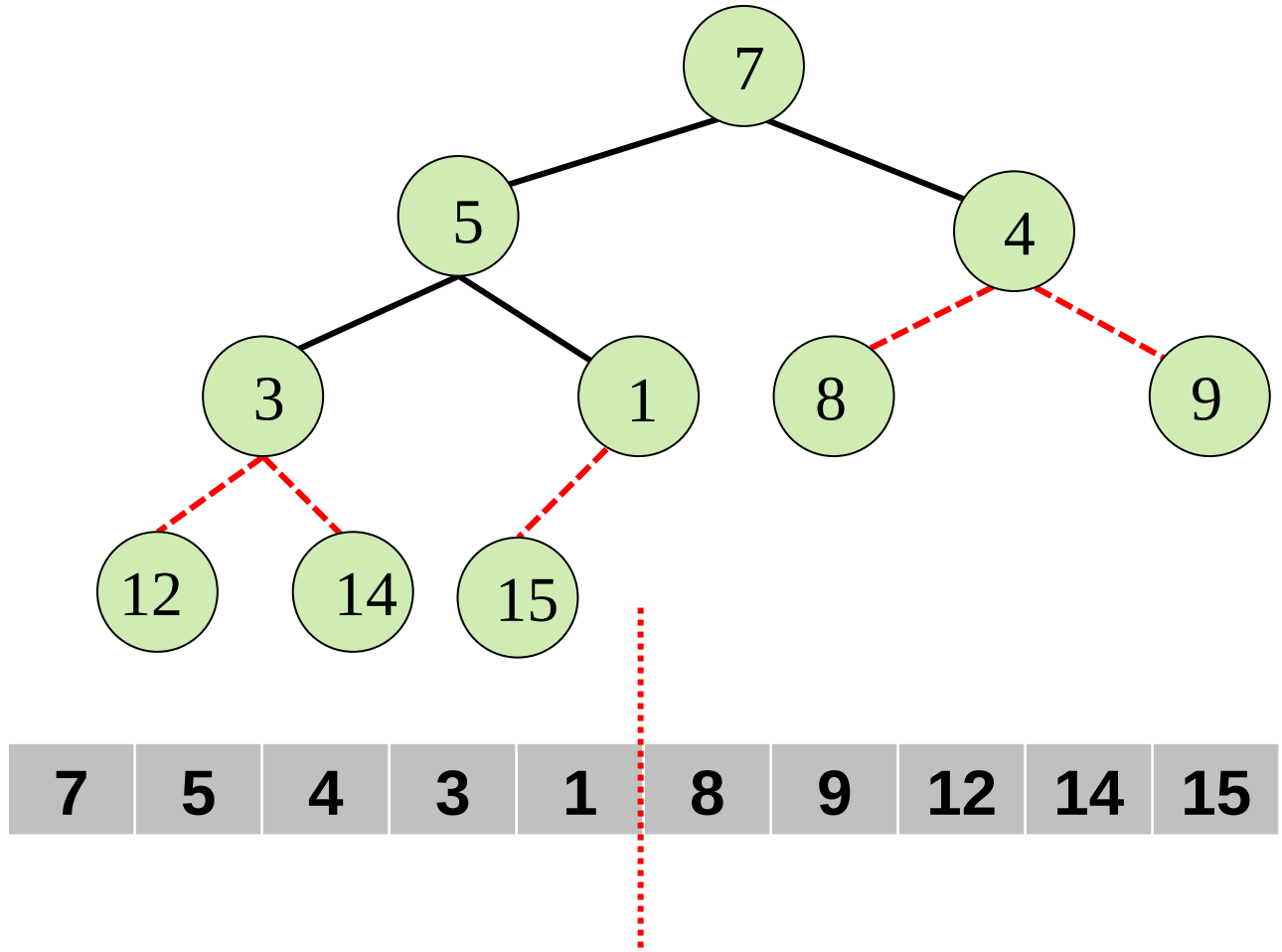
Trocar e descer



8	7	4	3	5	1	9	12	14	15

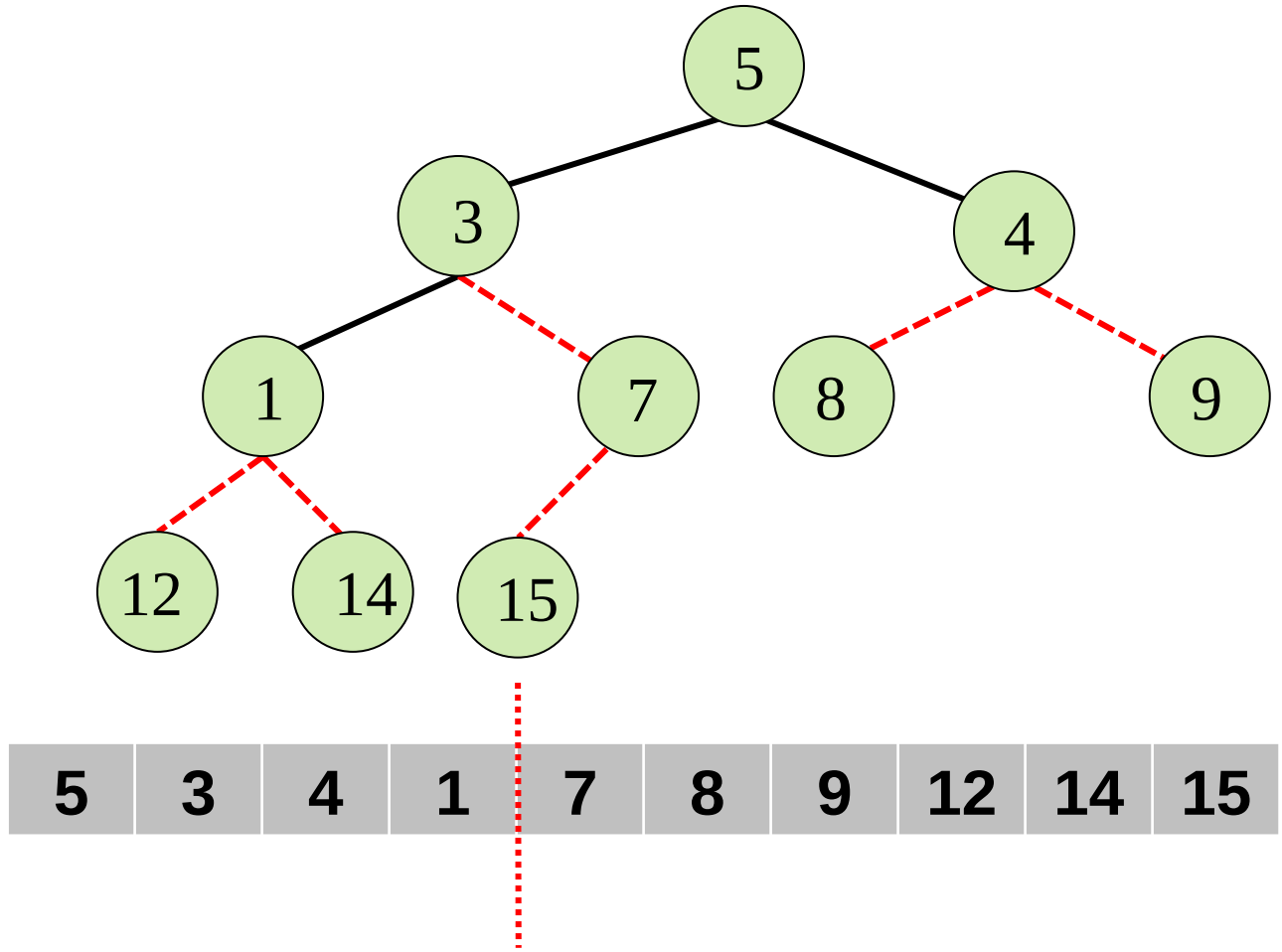
Heapsort

Trocar e descer



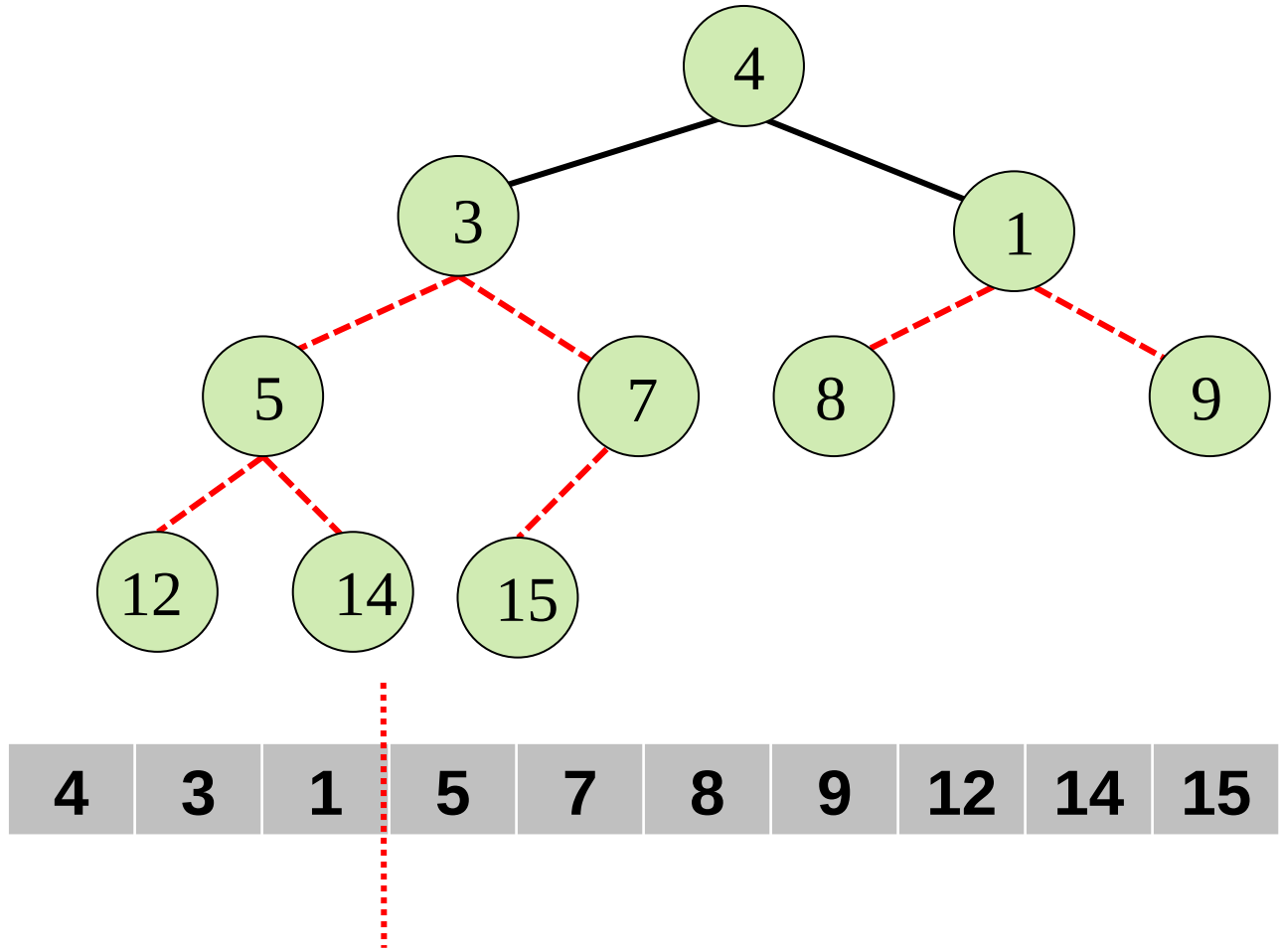
Heapsort

Trocar e descer



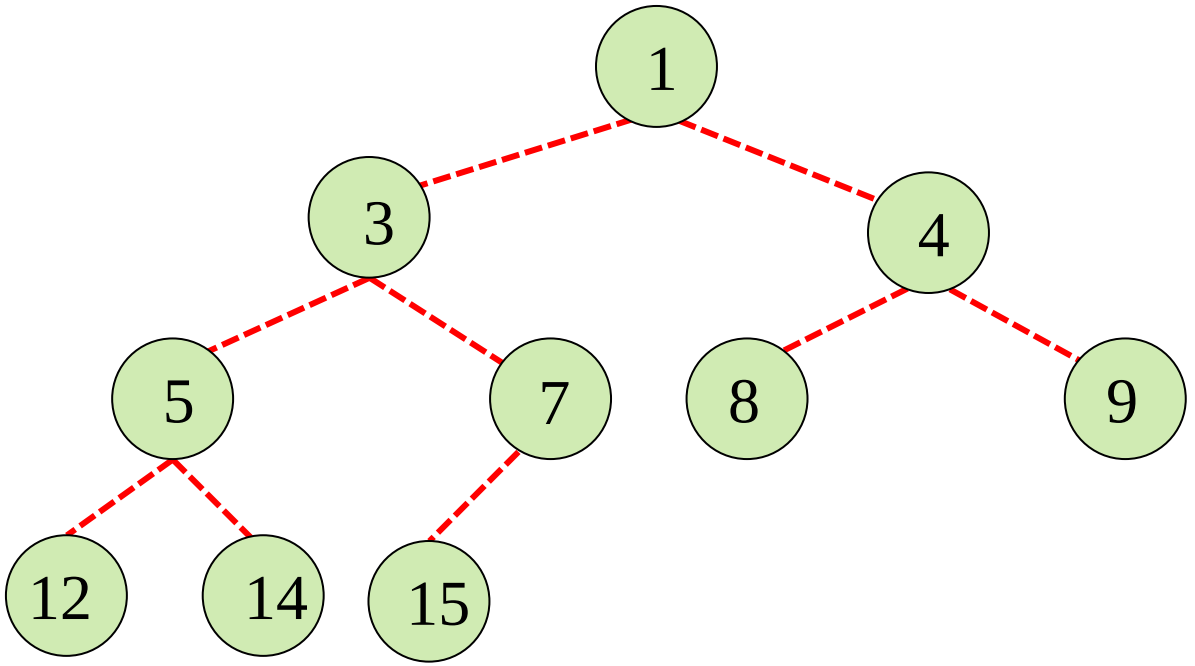
Heapsort

Trocar e descer



Heapsort

Final...



1	3	4	5	7	8	9	12	14	15

Exercício

1. Escrever os procedimentos subir e descer não recursivos. Qual a complexidade destes procedimentos?

